

Lab 04 — Highly Available & Fault-Tolerant Architecture

Fail over from a primary to a standby, measure the recovery time, restore the primary and take a database backup.

Lab environment: Docker Desktop (free local simulation) — runs locally, no AWS account needed

Estimated time: 35–40 minutes

SAA-C03 domain: Design Resilient Architectures

Overview

Local component	AWS equivalent
lab4-proxy (HAProxy, port 8084, health checks)	Route 53 failover routing / ELB health checks
lab4-primary	Primary site or instance
lab4-secondary (backup server)	Standby site or instance
lab4-db (MariaDB) + mariadb-dump	Amazon RDS + automated backups / snapshots

Step 1 — Get the lab source and start the lab

```
git clone https://github.com/mohanpothula/aws-saa.git
cd aws-saa/labs/lab-04-high-availability-fault-tolerance
docker compose up -d
docker compose ps
```

Step 2 — Verify the primary

Task: Access the HA endpoint.

```
curl http://localhost:8084
```

Expected result: PRIMARY is returned.

Step 3 — Simulate a failure and time the failover

Task: Stop the primary and time how long HAProxy takes to switch to the standby.

```
docker compose stop primary
time sh -c 'until curl -fs --max-time 3 http://localhost:8084 | grep -q SECONDARY; do sleep 1; done'
curl http://localhost:8084
```

Expected result: SECONDARY is returned. The **real** time printed by **time** is your recovery time (RTO) — about 8–10 seconds with the default health checks.

Step 4 — Explain and tune the RTO

Task: Find what sets the recovery time, then shorten it.

```
docker compose start primary
until curl -s http://localhost:8084 | grep -q PRIMARY; do sleep 1; done
sed -i 's/ check/ check inter 1s fall 2 rise 2/' haproxy.cfg
docker compose up -d --force-recreate proxy
sleep 3
docker compose stop primary
time sh -c 'until curl -fs --max-time 3 http://localhost:8084 | grep -q SECONDARY; do sleep 1; done'
```

Expected result: Failover is noticeably faster than in Step 3. RTO is set by the health checks — how often HAProxy checks (**inter**) times how many failures it waits for (**fall**) — not by the servers themselves.

Step 5 — Restore the primary

Task: Start the primary again.

```
docker compose start primary
sleep 3
curl http://localhost:8084
```

Expected result: The service is healthy.

Step 6 — Back up the database

Task: Create data and take a backup.

```
docker exec lab4-db mariadb -uroot -pPass123! -e "CREATE TABLE appdb.t(id INT); INSERT INTO
appdb.t VALUES(1);"
docker exec lab4-db mariadb-dump -uroot -pPass123! appdb > backup.sql
```

Expected result: **backup.sql** contains the **INSERT INTO** statement.

Verification

Check	Expected
Step 2 — Verify the primary	PRIMARY is returned
Step 3 — Simulate a failure and time the failover	SECONDARY is returned. The real time printed by time is your recovery time (RTO) — about 8–10 seconds with the default health checks
Step 4 — Explain and tune the RTO	Failover is noticeably faster than in Step 3. RTO is set by the health checks — how often HAProxy checks (inter) times how many failures it waits for (fall) — not by the servers themselves
Step 5 — Restore the primary	The service is healthy
Step 6 — Back up the database	backup.sql contains the INSERT INTO statement

Completion check

- [] Every validation result demonstrated to the trainer
- [] You can say which AWS service each local component represents
- [] You recorded an RTO and can explain what drives it

Cleanup

```
docker compose down -v
```

-v also removes the lab's volumes, so the next run starts from a clean state.

Reference

- Lab source code:
<https://github.com/mohanpothula/aws-saa/tree/main/labs/lab-04-high-availability-fault-tolerance>
- Original student guide: [courseware/AWS_SAA_Lab4_Student_Guide.docx](#)
- [Route 53 DNS failover](#)
- [Amazon RDS Multi-AZ deployments](#)

- [Amazon RDS automated backups](#)
- [Disaster Recovery of Workloads on AWS \(whitepaper\)](#)
- [HAProxy documentation](#)
- [MariaDB official Docker image](#)